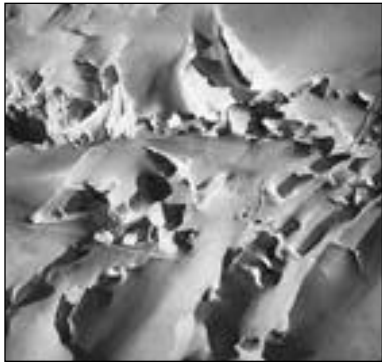


4



Requirements Elicitation

*Nobody is exempt from making mistakes.
The great thing is to learn from them.*

—Karl Popper, in *Objective Knowledge: An Evolutionary Approach*

A *requirement* is a feature that the system must have or a constraint that it must satisfy to be accepted by the client. *Requirements engineering* aims at defining the requirements of the system under construction. Requirements engineering includes two main activities; *requirements elicitation*, which results in the specification of the system that the client understands, and *analysis*, which results into an analysis model that the developers can unambiguously interpret. Requirements elicitation is the more challenging of the two because it requires the collaboration of several groups of participants with different backgrounds. On the one hand, the client and the users are experts in their domain and have a general idea of what the system should do. However, they often have little experience in software development. On the other hand, the developers have experience in building systems but often have little knowledge of the everyday environment of the users.

Scenarios and use cases provide tools for bridging this gap. A *scenario* describes an example of system use in terms of a series of interactions between the user and the system. A *use case* is an abstraction that describes a class of scenarios. Both scenarios and use cases are written in natural language, a form that is understandable to the user.

In this chapter, we focus on scenario-based requirements elicitation. Developers elicit requirements by observing and interviewing users. Developers first represent the user's current work processes as as-is scenarios and then develop visionary scenarios describing the functionality provided by the future system. The client and users validate the system description by reviewing the scenarios and by testing small prototypes provided by the developers. As the definition of the system matures and stabilizes, developers and the client agree on a system specification in the form of use cases.

4.1 Introduction: Usability examples

Consider the following examples¹:

Feet or miles?

During a laser experiment, a laser beam was directed at a mirror on the Space Shuttle Discovery. The test called for the laser beam to be reflected back toward a mountain top. The user entered the elevation of the mountain as “10,023”, assuming the units of the input were in feet. The computer interpreted the number in miles and the laser beam was reflected away from Earth, toward a hypothetical mountain 10,023 miles high.

Decimal point versus thousand separator

In the United States, decimal points are represented by a period (“.”) and thousand separators are represented by a comma (“,”). In Germany, the decimal point is represented by a comma and the thousand separator by a period. Assume a user in Germany, aware of both conventions, is viewing an on-line catalog with prices listed in dollars. Which convention should be used to avoid confusion?

Standard patterns

In the Emacs text editor, the command binding <Control-x><Control-q> exits the program. If any files need to be saved, the editor will ask the user, “Save file myDocument.txt? (y or n)”. If the user answers y, the editor saves the file prior to exiting. Many users rely on this pattern and systematically type the sequence <Control-x><Control-q>y when exiting an editor. Other editors, however, ask when exiting the question: “Are you sure you want to exit? (y or n)”. When users switch from Emacs to such an editor, they will fail to save their work until they manage to break this pattern.

Requirements elicitation is about communication among developers, clients, and users for defining a new system. Failure to communicate and understand each others’ domain results in a system that is difficult to use or that simply fails to support the user’s work. Errors introduced during requirements elicitation are expensive to correct, as they are usually discovered late in the process, often as late as delivery. Such errors include missing functionality that the system should have supported, functionality that was incorrectly specified, user interfaces that are misleading or unusable, and functionality that is obsolete. Requirements elicitation methods aim at improving communication among developers, clients, and users. Developers construct a model of the application domain by observing users in their environment (e.g., task analysis). Developers select a representation that is understandable by the clients and users (e.g., scenarios and use cases). Developers validate the application domain model by constructing simple prototypes of the user interface and collecting feedback from potential users (e.g., rapid prototyping, usability testing).

In the next section, we provide an overview of requirements elicitation and its relationship to the other software development activities. In Section 4.3, we define the main concepts used in this chapter. In Section 4.4, we discuss the activities of requirements elicitation. In Section 4.5, we discuss the management activities related to requirements elicitation.

1. Examples taken and adapted from [Nielsen, 1993] and from the RISK forum.

4.2 An overview of requirements elicitation

Requirements elicitation focuses on describing the purpose of the system. The client, the developers, and the users identify a problem area and define a system that addresses the problem. Such a definition is called a **system specification** and serves as a contract between the client and the developers. The system specification is structured and formalized during analysis (Chapter 5, *Analysis*) to produce an analysis model (see Figure 4-1). Both system specification and analysis model represent the same information. They differ only in the language and notation they use. The system specification is written in natural language, whereas the analysis model is usually expressed in a formal or semiformal notation. The system specification supports the communication with the client and users. The analysis model supports the communication among developers. They are both models of the system in the sense that they attempt to accurately represent the external aspects of the system. Given that both models represent the same aspects of the system, requirements elicitation and analysis occur concurrently and iteratively.

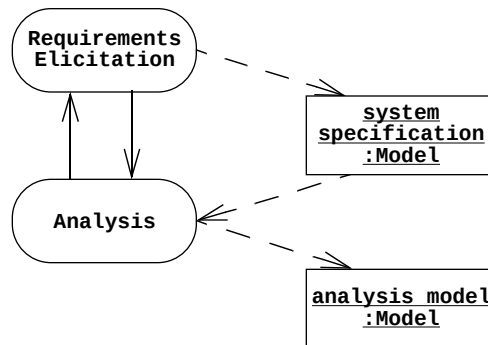


Figure 4-1 Products of requirements elicitation and analysis (UML activity diagram).

Requirements elicitation and analysis focus only on the user's view of the system. For example, the system functionality, the interaction between the user and the system, the errors that the system can detect and handle, and the environmental conditions in which the system functions, are part of the requirements. The system structure, the implementation technology selected to build the system, the system design, the development methodology, and other aspects not directly visible to the user are not part of the requirements.

Requirements elicitation includes the following activities.

- **Identifying actors.** During this activity, developers identify the different types of users the future system will support.
- **Identifying scenarios.** During this activity, developers observe users and develop a set of detailed scenarios for typical functionality provided by the future system. Scenarios are

concrete examples of the future system in use. Developers use these scenarios to communicate with the user and deepen their understanding of the application domain.

- **Identifying use cases.** Once developers and users agree on a set of scenarios, developers derive from the scenarios a set of use cases that completely represent the future system. Whereas scenarios are concrete examples illustrating a single case, use cases are abstractions describing all possible cases. When describing use cases, developers determine the scope of the system.
- **Refining use cases.** During this activity, developers ensure that the system specification is complete, by detailing each use case and describing the behavior of the system in the presence of errors and exceptional conditions.
- **Identifying relationships among use cases.** During this activity, developers consolidate the use case model by eliminating redundancies. This ensures that the system specification is consistent.
- **Identifying nonfunctional requirements.** During this activity, developers, users, and clients agree on aspects that are visible to the user but not directly related to functionality. These include constraints on the performance of the system, its documentation, the resources it consumes, its security, and its quality.

During requirements elicitation, developers access many different sources of information, including client-supplied documents about the application domain, manuals and technical documentation of legacy systems that the future system will replace, and, most important, the users and clients themselves. Developers interact the most with users and clients during requirements elicitation. We focus on three methods for eliciting information and making decisions with users and clients:

- **Joint Application Design (JAD)** focuses on building consensus among developers, users, and clients by jointly developing the system specification.
- **Knowledge Analysis of Tasks (KAT)** focuses on eliciting information from users through observation.
- **Usability testing** focuses on validating the requirements elicitation model with the user through a variety of methods.

4.3 Requirements elicitation concepts

In this section, we describe the main requirements elicitation concepts we use in this chapter. In particular, we describe:

- functional requirements (Section 4.3.1)
- nonfunctional and pseudorequirements (Section 4.3.2)
- levels of descriptions (Section 4.3.3)
- correctness, completeness, consistency, clarity, and realism (Section 4.3.4)

- verifiability and traceability (Section 4.3.5)
- greenfield engineering, reengineering, and interface engineering (Section 4.3.6)

We describe the requirements elicitation activities in Section 4.4.

4.3.1 Functional requirements

Functional requirements describe the interactions between the system and its environment independent of its implementation. The environment includes the user and any other external system with which the system interacts. For example, the following is an example of functional requirements for SatWatch, a watch that resets itself without user intervention:

Functional requirements for SatWatch

SatWatch is a wrist watch that displays the time based on its current location. SatWatch uses GPS satellites (Global Positioning System) to determine its location and internal data structures to convert this location into a time zone. The information stored in the watch and its accuracy measuring time (one hundredth of second uncertainty over five years) is such that the watch owner never needs to reset the time. SatWatch adjusts the time and date displayed as the watch owner crosses time zones and political boundaries (e.g., standard time vs. daylight savings time). For this reason, SatWatch has no buttons or controls available to the user.

SatWatch has a two-line display showing, on the top line, the time (hour, minute, second, time zone) and, on the bottom line, the date (day of the week, day, month, year). The display technology used is such that the watch owner can see the time and date even under poor light conditions.

When a new country or state institutes different rules for daylight savings time, the watch owner may upgrade the software of the watch using the WebifyWatch serial device (provided when the watch is purchased) and a personal computer connected to the Internet. SatWatch complies with the physical, electrical, and software interfaces defined by WebifyWatch API 2.0.

The above functional requirements only focus on the possible interactions between SatWatch and its external world (i.e., the watch owner, GPS, and WebifyWatch). The above description does not focus on any of the implementation details (e.g., processor, language, display technology).

4.3.2 Nonfunctional and pseudorequirements

Nonfunctional requirements describe user-visible aspects of the system that are not directly related with the functional behavior of the system. Nonfunctional requirements include quantitative constraints, such as response time (i.e., how fast the system reacts to user commands) or accuracy (i.e., how precise are the system's numerical answers). The following are the nonfunctional requirements for SatWatch:

Nonfunctional requirements for SatWatch

SatWatch determines its location using GPS satellites, and as such, suffers from the same limitations as all other GPS devices (e.g., ~100 feet accuracy, inability to determine location at certain times of the day in mountainous regions). During blackout periods, SatWatch assumes that it does not cross a time zone or a political boundary. SatWatch corrects its time zone as soon as a blackout period ends.

The battery life of SatWatch is limited to 5 years, which is the estimated life cycle of the housing of SatWatch. The SatWatch housing is not designed to be opened once manufactured, preventing battery replacement and repairs. Instead, SatWatch is priced such that the watch owner is expected to buy a new SatWatch to replace a defective or old SatWatch.

Pseudorequirements are requirements imposed by the client that restrict the implementation of the system. Typical pseudorequirements are the implementation language and the platform on which the system is to be implemented. For life-critical developments, pseudorequirements often include process and documentation requirements (e.g., the use of a formal specification method, the complete release of all work products). Pseudorequirements have usually no direct effect on the users' view of the system. The following are the pseudofunctional requirements for SatWatch:

Pseudorequirement for SatWatch

All related software associated with SatWatch, including the onboard software, will be written using Java, to comply with current company policy.

Analysis is a modeling activity. The developer constructs a model describing the reality as seen from a user's point of view. Modeling consists of identifying and classifying real-world phenomena (e.g., aspects of the system under construction) into concepts. Figure 4-2 is a UML class diagram representing the relationships between models and reality. In this diagram, a model is said to be correct if each concept in the model corresponds to a relevant phenomenon. The model is complete if all relevant phenomena are represented by at least one concept. The model is consistent if all concepts represent phenomena of the same reality (i.e., if a model is inconsistent, it must represent aspects of two different realities).

4.3.3 Levels of description

Requirements describe a system and its interaction with the surrounding environment, such as the users, their work processes, and other systems. Most requirements analysis methods have focused on describing the system. When using use cases and scenarios, it becomes apparent, however, that it is also necessary to describe the environment in which the system will operate. First, developers usually do not initially know and understand the operating environment and need to check their understanding with the users. Second, the environment is likely to change, and thus, developers should capture all the assumptions they make about the environment. In general, there are four levels of description, which can uniformly be described with use cases [Paech, 1998]. We list them below from most-general to most-specific:

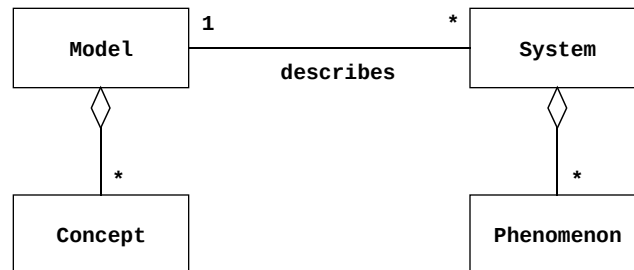


Figure 4-2 A System is a collection of real world Phenomena. A model is a collection of concepts that represent the system's phenomena. Many models can represent different aspects of the same system. An unambiguous model corresponds to only one system.

- **Work division.** This set of use cases describes the work processes of the users that are relevant to the system. The part of the process supported by the system is also described, but the focus is on defining the boundaries between the users and the system.
- **Application-specific system functions.** This set of use cases describes the functions that the system provides that are related to the application domain.
- **Work-specific system functions.** This set of use cases describes the supporting functions of the system that are not directly related with the application domain. These include file management functions, grouping functions, undo functions, and so on. These use cases will be extended during system design, when we are discussing known boundary conditions, such as system initialization, shutdown, and exception handling policies.
- **Dialog.** This set of use cases describes the interactions between the users and the user interface of the system. The focus is on designing resolving control flow and layout issues.

4.3.4 Correctness, completeness, consistency, clarity, and realism

Requirements are continuously validated with the client and the user. Validation is a critical step in the development process, given that both the client and the developer are dependent on the system specification. Requirement validation involves checking if the specification is correct, complete, consistent, unambiguous, and realistic. A specification is **correct** if it represents the client's view of the system (i.e., everything in the requirements model accurately represents an aspect of the system). It is **complete** if all possible scenarios through the system are described, including exceptional behavior (i.e., all aspects of the system are represented in the requirements model). The system specification is **consistent** if it does not contradict itself. The system specification is **unambiguous** if exactly one system is defined (i.e., it is not possible to interpret the specification two or more different ways). Finally, it is **realistic** if the system can be implemented within constraints. These properties are illustrated with UML instance diagrams in Table 4-1.

Table 4-1 Specification properties checked during validation

Correct —The model describes the reality of interest to the client, not another reality	
Complete —Every phenomenon of interest is described in the model by a concept	
Consistent —All concepts in the model correspond to phenomena of the same reality	
Unambiguous —All concepts in the model correspond to exactly one phenomenon	
Realistic —The model describes a reality that can exist	

The correctness and completeness of a system specification are often difficult to establish, especially before the system exists. Given that the system specification serves as a contractual basis between the client and the developers, the system specification must be carefully reviewed by both parties. Additionally, parts of the system that present a high risk should be prototyped or simulated to demonstrate their feasibility or to obtain feedback from the user. In the case of SatWatch described above, a mock-up of the watch would be built using a traditional watch and users surveyed to gather their initial impressions. A user may remark that she wants the watch to be able to display both American and European date formats.

4.3.5 Verifiability and traceability

Two more desirable properties of a system specification are that it be verifiable and traceable. The specification is **verifiable** if, once the system is built, a repeatable test can be designed to demonstrate that the system fulfills the requirement. For example, a mean time to failure of a hundred years for SatWatch would be difficult to achieve (assuming it is realistic in the first place). The following requirements are additional examples of nonverifiable requirements:

- *The product shall have a good user interface* (good is not defined).
- *The product shall be error free* (requires large amount of resources to establish).
- *The product shall respond to the user with 1 second for most cases* (“most cases” is not defined).

A system specification is **traceable** if each system function can be traced to its corresponding set of requirements. Traceability is not a constraint on the content of the specification, but rather, on its organization. Traceability facilitates the development of tests and the systematic validation of the design against the requirements.

4.3.6 Greenfield engineering, reengineering, and interface engineering

Requirements elicitation activities can be classified into three categories, depending on the source of the requirements. In **greenfield engineering**, the development starts from scratch, no prior system exists, the requirements are extracted from the users and the client. A greenfield engineering project is triggered by a user need or the creation of a new market. SatWatch is a greenfield engineering project.

A **reengineering** project is the redesign and reimplementation of an existing system triggered by technology enablers or by new information flow [Hammer & Champy, 1993]. Sometimes, the functionality of the new system is extended; however, the essential purpose of the system remains the same. The requirements of the new system are extracted from an existing system.

An **interface engineering** project is the redesign of the user interface of an existing system. The legacy system is left untouched except for its interface, which is redesigned and

reimplemented. This type of project is a reengineering project in which the legacy system cannot be discarded without entailing high costs. In this section, we examine how requirements elicitation is performed in all three situations.

In both reengineering and greenfield engineering, the developers need to gather as much information as possible from the application domain. This information can be found in procedures manuals, documentation distributed to new employees, the previous system's manual, glossaries, cheat sheets and notes developed by the users, and user and client interviews. Note that although interviews with users are an invaluable tool, they fail to gather the necessary information if the relevant questions are not asked. Developers must first gain a solid knowledge of the application domain before the direct approach can be used.

Next, we describe the activities of requirements elicitation that result in a system specification.

4.4 Requirements elicitation activities

In this section, we describe the requirements elicitation activities. These map a problem statement into a system specification that we represent as a set of actors, scenarios, and use cases (see Chapter 2, *Modeling with UML*). We discuss heuristics and methods for extracting requirements from users and modeling the system in terms of these concepts. Requirements elicitation activities include:

- identifying actors (Section 4.4.1)
- identifying scenarios (Section 4.4.2)
- identifying use cases (Section 4.4.3)
- refining use cases (Section 4.4.4)
- identifying relationships among use cases (Section 4.4.5)
- identifying participating objects (Section 4.4.6)
- identifying nonfunctional requirements (Section 4.4.7)

The methods described in this section are adapted from OOSE [Jacobson et al., 1992], the Unified Software Development Process [Jacobson et al., 1999], and Responsibility-driven design [Wirfs-Brock et al., 1990].

4.4.1 Identifying actors

Actors represent external entities that interact with the system. An actor can be human or an external system. In the SatWatch example, the watch owner, the GPS satellites, and the WebifyWatch serial device are actors (see Figure 4-3). They all interact and exchange information with the SatWatch. Note, however, they all have specific interactions with the SatWatch: the watch owner wears and looks at her watch; the watch monitors the signal from the GPS satellites; the WebifyWatch downloads new data into the watch. Actors define classes of functionality.

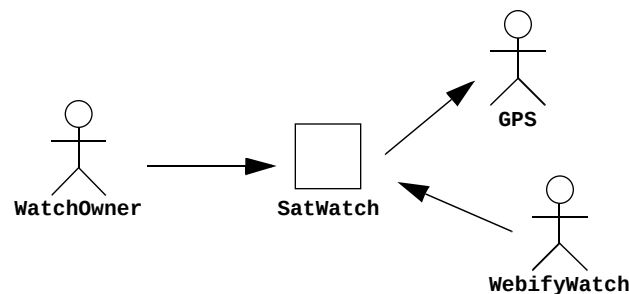


Figure 4-3 Actors for the SatWatch system. WatchOwner moves the watch (possibly across time zones) and consults it to know what time it is. SatWatch interacts with GPS to compute its position. WebifyWatch upgrades the data contained in the watch to reflect changes in time policy (e.g., changes in daylight savings time start and end dates).

Consider a more-complex example, FRIEND, a distributed information system for accident management [FRIEND, 1994], [Bruegge et al., 1994]. It includes many actors, such as FieldOfficer, who represents the police and fire officers who are responding to an incident, and Dispatcher, the police officer responsible for answering 911 calls and dispatching resources to an incident. FRIEND supports both actors by keeping track of incidents, resources, and task plans. It also has access to multiple databases, such as a hazardous materials database and emergency operations procedures. The FieldOfficer and the Dispatcher actors interact through different interfaces: FieldOfficers access FRIEND through a mobile personal assistant, Dispatchers access FRIEND through a workstation (see Figure 4-4).

Actors are role abstractions and do not necessarily directly map to persons. The same person can fill the role of FieldOfficer or Dispatcher at different times. However, the functionality they access is substantially different. For that reason, these two roles are modeled as two different actors.

The first step of requirements elicitation is the identification of actors. This serves both to define the boundaries of the system and to find all the perspectives from which the developers need to consider the system. When the system is deployed into an existing organization (such as a company), most actors usually exist before the system is developed: they correspond to roles in the organization.

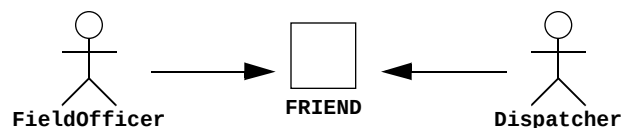


Figure 4-4 Actors of the FRIEND system. FieldOfficers not only have access to different functionality, they use different computers to access the system.

When identifying actors, developers can ask the following questions:

Questions for identifying actors

- Which user groups are supported by the system to perform their work?
- Which user groups execute the system's main functions?
- Which user groups perform secondary functions, such as maintenance and administration?
- Will the system interact with any external hardware or software system?

In the FRIEND example, these questions lead to a long list of potential actors: Fire Fighter, Police Officer, Dispatcher, Investigator, Mayor, Governor, an EPA hazardous material database, System Administrator, and so on. We then need to consolidate this list into a small number of actors, who are different from the point of view of the usage of the system. For example, a Fire Fighter and a Police Officer may share the same interface to the system as they are both involved with a single incident in the field. A Dispatcher, on the other hand, manages multiple concurrent incidents and requires access to a larger amount of information. The Mayor and the Governor will not likely interact directly with the system and will use the services of a trained Operator instead.

Once the actors are identified, the next step in the requirements elicitation activity is to determine the functionality that is accessible to each actor. This information can be extracted using scenarios and formalized using use cases.

4.4.2 Identifying scenarios

A scenario is “a narrative description of what people do and experience as they try to make use of computer systems and applications” [Carroll, 1995]. A scenario is a concrete, focused, informal description of a single feature of the system from the viewpoint of a single actor. The use of scenarios in requirements elicitation is a conceptual departure from the traditional representations, which are generic and abstract. Traditional representations are centered around the system as opposed to the work that the system supports. Finally, their focus is on completeness, consistency, and accuracy, whereas scenarios are open ended and informal. A scenario-based approach cannot (and is not intended to) completely replace traditional approaches. It does, however, enhance requirements elicitation by providing a tool that is readily understandable to users and clients.

Figure 4-5 is an example of scenario for the FRIEND system [FRIEND, 1994], an information system for incident response. In this scenario, a police officer reports a fire and a Dispatcher initiates the incident response. Note that this scenario is concrete, in the sense that it describes a single instance. It does not attempt to describe all possible situations in which a fire incident is reported.

<i>Scenario name</i>	warehouseOnFire
<i>Participating actor instances</i>	bob, alice: FieldOfficer john: Dispatcher
<i>Flow of events</i>	1. Bob, driving down main street in his patrol car notices smoke coming out of a warehouse. His partner, Alice, activates the “Report Emergency” function from her FRIEND laptop. 2. Alice enters the address of the building, a brief description of its location (i.e., northwest corner), and an emergency level. In addition to a fire unit, she requests several paramedic units on the scene, given that the area appears to be relatively busy. She confirms her input and waits for an acknowledgment. 3. John, the Dispatcher, is alerted to the emergency by a beep of his workstation. He reviews the information submitted by Alice and acknowledges the report. He creates allocates a fire unit and two paramedic units to the Incident site and sends their estimated arrival time (ETA) to Alice. 4. Alice receives the acknowledgment and the ETA.

Figure 4-5 warehouseOnFire scenario for the ReportEmergency use case.

Scenarios can have many different uses during requirements elicitation and during other activities of the life cycle. Below is a selected number of scenario types taken from [Carroll, 1995]:

- *As-is scenarios* describe a current situation. During reengineering, for example, the current system is understood by observing users and describing their actions as scenarios. These scenarios can then be validated for correctness and accuracy with the users.
- *Visionary scenarios* describe a future system, either re-engineered or designed from scratch. Visionary scenarios are used both as a design representation by developers as they refine their idea of the future system and as a communication medium to elicit requirements from users. Visionary scenarios can be viewed as an inexpensive prototype.
- *Evaluation scenarios* describe user tasks against which the system is to be evaluated. The collaborative development of evaluation scenarios by users and developers also improves the definition of the functionality tested by these scenarios.
- *Training scenarios* are tutorials used for introducing new users to the system. These are step by step instructions designed to hand-hold the user through common tasks.

In requirements elicitation, developers and users write and refine a series of scenarios in order to gain a shared understanding of what the system should be. Initially, each scenario may be high level and incomplete, as the warehouseOnFire scenario is. The following questions can be used for identifying scenarios.

Questions for identifying scenarios

- What are the tasks that the actor wants the system to perform?
- What information does the actor access? Who creates that data? Can it be modified or removed? By whom?
- Which external changes does the actor need to inform the system about? How often? When?
- Which events does the actor need to be informed by the system about? With what latency?

Developers use existing documents about the application domain to answer these questions. These documents include user manuals of previous systems, procedures manuals, company standards, user notes and cheat sheets, user and client interviews. Developers should always write scenarios using application domain terms, as opposed to their own terms. As developers gain further insight in the application domain and the possibilities of the available technology, they iteratively and incrementally refine scenarios to include increasing amounts of detail. Drawing user interface mock-ups often helps to find omissions in the specification and help the users build a more concrete picture of the system.

In the FRIEND example, we may identify four scenarios that span the type of tasks the system is expected to support:

- **warehouseOnFire** (Figure 4-5): A fire is detected in a warehouse; two field officers arrive at the scene and request resources.
- **fenderBender** A car accident without casualties occurs on the highway. Police officers document the incident and manage traffic while the damaged vehicles are towed away.
- **catInATree** A cat is stuck in a tree. A fire truck is called to retrieve the cat. Because the incident is low priority, the fire truck takes time to arrive at the scene. In the mean time, the impatient cat owner climbs the tree, falls, and breaks a leg, requiring an ambulance to be dispatched.
- **earthQuake** An unprecedented earthquake seriously damages buildings and roads, spanning multiple incidents and triggering the activation of a statewide emergency operations plan. The governor is notified. Road damage hampers incident response.

The emphasis of actor identification and scenario identification is for developers to understand the application domain and to define the right system. This results in a shared understanding of the user work processes that need to be supported and of the scope of the system. Once developers identified and described actors and scenarios, developers formalize scenarios into use cases.

4.4.3 Identifying use cases

A scenario is an instance of a use case, that is, a use case specifies all possible scenarios for a given piece of functionality. A use case is initiated by an actor. After its initiation, a use case may interact with other actors as well. A use case represents a complete flow of events

<i>Use case name</i>	ReportEmergency
<i>Participating actor</i>	Initiated by FieldOfficer Communicates with Dispatcher
<i>Entry condition</i>	1. The FieldOfficer activates the “Report Emergency” function of her terminal.
<i>Flow of events</i>	2. FRIEND responds by presenting a form to the officer. 3. The FieldOfficer fills the form, by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form, at which point the Dispatcher is notified. 4. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the emergency report.
<i>Exit condition</i>	5. The FieldOfficer receives the acknowledgment and the selected response.
<i>Special requirements</i>	The FieldOfficer’s report is acknowledged within 30 seconds. The selected response arrives no later than 30 seconds after it is sent by the Dispatcher.

Figure 4-6 An example of use case: ReportEmergency.

through the system in the sense that it describes a series of related interactions that result from the initiation of the use case.

Figure 4-6 depicts the use case ReportEmergency of which the scenario warehouseOnFire (see Figure 4-5) is an instance. The FieldOfficer actor initiates this use case by activating the “Report Emergency” function of FRIEND. The use case completes when the FieldOfficer actor receives an acknowledgment that an incident has been created. This use case is general and encompasses a range of scenarios. For example, the ReportEmergency use case could also apply to the fenderBenders scenario. Use cases can be written at varying levels of detail as in the case of scenarios. The ReportEmergency use case may be illustrative enough to describe how FRIEND supports reporting emergencies and to obtain general feedback from the user, but it does not provide sufficient detail for a system specification.

4.4.4 Refining use cases

Figure 4-7 is a refined version of the ReportEmergency use case. It has been extended to include details about the type of incidents that are known to FRIEND, detailed interactions indicating how the Dispatcher acknowledges the FieldOfficer

<i>Location</i>	<i>Use case description</i>
<i>FieldOfficer station</i>	1. The FieldOfficer activates the “Report Emergency” function of her terminal. 2. FRIEND responds by presenting a form to the officer. The form includes an emergency type menu (General emergency, fire, transportation), a location, incident description, resource request, and hazardous material fields. 3. The FieldOfficer fills the form by specifying minimally the emergency type and description fields. The FieldOfficer may also describes possible responses to the emergency situation and request specific resources. Once the form is completed, the FieldOfficer submits the form by pressing the “Send Report” button, at which point the Dispatcher is notified.
<i>Dispatcher station</i>	4. The Dispatcher is notified of a new incident report by a popup dialog. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. All the information contained in the FieldOfficer’s form is automatically included in the Incident. The Dispatcher selects a response by allocating resources to the Incident (with the AllocateResources use case) and acknowledges the emergency report by sending a short message to the FieldOfficer.
<i>FieldOfficer station</i>	5. The FieldOfficer receives the acknowledgment and the selected response.

Figure 4-7 Refined description for the ReportEmergency use case.

The use of scenarios and use cases to define the functionality of the system aims at creating requirements that are validated by the user early in the development. As the design and implementation of the system starts, the cost of changing the system specification and adding new unforeseen functionality increases. Although requirements change until late in the development, developers and users should strive to address most requirements issues early. This entails lots of changes and experimentation during requirements elicitation. Note that many use cases are rewritten several times, others substantially refined, and yet others completely dropped. In order to save time, a lot of the exploration work can be done using scenarios and user interface mock-ups. The following heuristics can be used for writing scenarios and use cases.

Heuristics for writing scenarios and use cases

- Use scenarios to communicate with users and to validate functionality.
- First, refine a narrow vertical slice (i.e., one scenario) to understand the user's preferred style of interaction.
- Next, define a horizontal slice (i.e., many not-very-detailed scenarios) to define the scope of the system. Validate with the user.
- Use mock-ups as a visual support only; user interface design should occur as a separate task once the functionality is sufficiently stable.
- Present the user with multiple alternatives (as opposed to extracting a single alternative from the user).
- Detail a broad vertical slice when the scope of the system and the user preferences are well understood. Validate with the user.

The focus of this activity is on completeness and correctness. Developers identify functionality not covered by scenarios and document it with new use cases. Developers describe rarely occurring cases and exception handling as seen by the actors. If actors require an on-line help support system, developers describe it with use cases during this activity.

Once the system specification becomes stable, traceability and redundancy issues can be addressed by consolidating and reorganizing the actors and use cases.

4.4.5 Identifying relationships among actors and use cases

Even medium-sized systems have many use cases. Relationships among actors and use cases enable the developers and users to reduce the complexity of the model and increase its understandability. We use communication relationships between actors and use cases to describe the system in layers of functionality. We use extend relationships to separate exceptional and common flows of events. We use include relationships to reduce redundancy among use cases.

Communication relationships between actors and use cases

Communication relationships between actors and use cases represent the flow of information during the use case. The actor who initiates the use case should be distinguished from the other actors with whom the use case communicates. Thus, access control (i.e., which actor has access to which class functionality) can be represented at this level. The relationships between actors and use cases are identified when use cases are identified. Figure 4-8 depicts an example of communication relationships in the case of the FRIEND system.

Extend relationships between use cases

A use case extends another use case if the extended use case may include the behavior of the extension under certain conditions. In the FRIEND example, assume that the connection between the FieldOfficer station and the Dispatcher station is broken while the FieldOfficer is filling the form (e.g., the FieldOfficer's car enters a tunnel). The FieldOfficer station needs to notify the FieldOfficer that his form was not delivered and what measures he should take. The ConnectionDown use case is modeled as an extension of

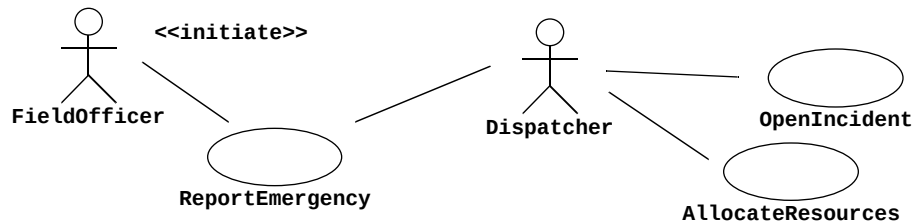


Figure 4-8 Example of communication relationships among actors and use cases in FRIEND (UML use case diagram). The FieldOfficer initiates the ReportEmergency use case and the Dispatcher initiates the OpenIncident and AllocateResources use cases. FieldOfficers cannot directly open an incident or allocate resources on their own.

ReportEmergency (see Figure 4-9). The conditions under which the ConnectionDown use case is initiated are described in ConnectionDown as opposed to ReportEmergency. Separating exceptional and optional flow of events from the base use case has two advantages. First, the base use case becomes shorter and easier to understand. Second, the common case is distinguished from the exceptional case, which enables the developers to treat each type of functionality differently (e.g., optimize the common case for response time, optimize the exceptional case for clarity). Both the extended use case and the extensions are complete use cases of their own. They must have an entry and an end condition and be understandable by the user as an independent whole.

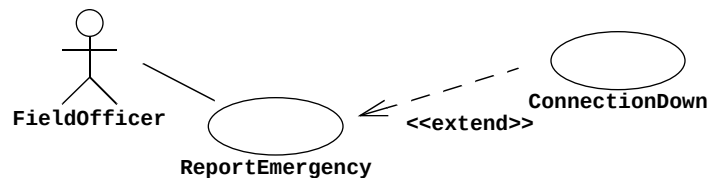


Figure 4-9 Example of use of extend relationship (UML use case diagram). ConnectionDown extends the ReportEmergency use case. The ReportEmergency use case becomes shorter and solely focused on emergency reporting.

Include relationships between use cases

Redundancies among use cases can be factored out using include relationships. Assume, for example, that a Dispatcher needs to consult the city map when opening an incident (e.g., in order to assess which areas are at risk during a fire) and when allocating resources (e.g., to find which resources are closer to the incident). In this case, the ViewMap use case describes the flow of events required when viewing the city map and is used by both the OpenIncident and the AllocateResources use cases (Figure 4-10).

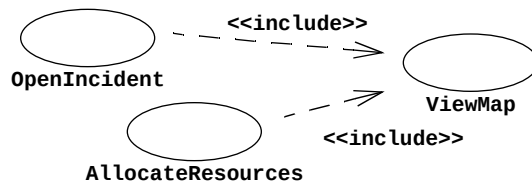


Figure 4-10 Example of include relationships among use cases. **ViewMap** describes the flow of events for viewing a city map (e.g., scrolling, zooming, query by street name) and is used by both **OpenIncident** and **AllocateResources** use cases.

Factoring out shared behavior from use cases has many benefits, including shorter descriptions and fewer redundancies. Behavior should *only* be factored out into a separate use case if it is shared across two or more use cases. Excessive fragmentation of the system specification across a large number of use cases makes the specification confusing to users and clients.

Extend versus include relationships

Include and extend are similar constructs, and initially it may not be clear to the developer when to use each construct [Jacobson et al., 1992]. The main distinction between these constructs is the direction of the relationship. In the case of an include relationship, the conditions under which the target use case is initiated are described in the initiating use case, as an event in the flow of events. In the case of an extend relationship, the conditions under which the extension is initiated are described in the extension as an entry condition. Figure 4-11 shows the **ConnectionDown** example described with an include relationship (left column) and with an extend relationship (right column). In the left column, we need to insert text in two places in the event flow where the **ConnectionDown** use case can be invoked. Also, if additional exceptional situations are described (e.g., a **Help** function on the **FieldOfficer** station), the **ReportEmergency** use case will have to be modified and will become cluttered with conditions. In the right column, we only need to describe the conditions under which the use case is invoked. Moreover, additional exceptional situations can be added with modifying the base use case (e.g., **ReportEmergency**). The ability to extend the system without modifying existing parts is critical, as it allows us to ensure that the original behavior is left untouched.

In summary, the following heuristics can be used for selecting an extend or an include relationship.

Heuristics for extend and include relationships

- Use extends for exceptional, optional, or seldom-occurring behavior.
- Use uses for behavior that is shared across two or more use cases.

ReportEmergency (include relationship)	ReportEmergency (extend relationship)
1. ... 2. ... 3. The FieldOfficer fills the form, by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form, at which point, the Dispatcher is notified. <i>If the connection with the Dispatcher is broken, the ConnectionDown use case is used.</i> 4. If the connection is still alive, the Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the emergency report. <i>If the connection is broken, the ConnectionDown use case is used.</i> 5. ...	1. ... 2. ... 3. The FieldOfficer fills the form, by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form, at which point, the Dispatcher is notified. 4. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response and acknowledges the emergency report. 5. ...
ConnectionDown (include relationship)	ConnectionDown (extend relationship)
1. The FieldOfficer and the Dispatcher are notified that the connection is broken. They are advised of the possible reasons why such an event would occur (e.g., "Is the FieldOfficer station in a tunnel?"). 2. The situation is logged by the system and recovered when the connection is reestablished. 3. The FieldOfficer and the Dispatcher enter in contact through other means and the Dispatcher initiates ReportEmergency from the Dispatcher station.	<i>The ConnectionDown use case extends ReportEmergency when the connection between the FieldOfficer and the Dispatcher is lost.</i> 1. The FieldOfficer and the Dispatcher are notified that the connection is broken. They are advised of the possible reasons why such an event would occur (e.g., "Is the FieldOfficer station in a tunnel?"). 2. The situation is logged by the system and recovered when the connection is reestablished. 3. The FieldOfficer and the Dispatcher enter in contact through other means and the Dispatcher initiates ReportEmergency from the Dispatcher station.

Figure 4-11 Addition of ConnectionDown exceptional condition to ReportEmergency. An extend relationship is used for exceptional and optional flow of events as it yields a more modular description.

In all cases, the purpose of adding include and extend relationships is to reduce or remove redundancies from the use case model, thus eliminating potential inconsistencies.

4.4.6 Identifying initial analysis objects

One of the first obstacles developers and users encounter when collaborating is differing terminology. Misunderstandings result from the same terms being used in different context and with different meanings. Although developers eventually learn the users' terminology, this problem is likely to be encountered again when new developers are added to the project.

Once use cases have been consolidated, developers identify the **participating objects** for each use case. The participating objects correspond to the main concepts in the application domain. Developers identify, name, and describe them unambiguously and collate them into a glossary.

This glossary is included in the system specification and, later, in the user manuals. Developers keep this glossary up to date as the system specification evolves. The benefits of the glossary are many: New developers are exposed to a consistent set of definitions, a single term is used for each concept (instead of a developer term and a user term), and each term has a precise and clear official meaning.

The identification of participating objects results in the initial analysis model. The identification of participating objects during requirements elicitation only constitute a first step toward the complete analysis model. The complete analysis model is usually not used as means of communication between users and developers, as users are often unfamiliar with object-oriented concepts. However, the description of the objects (i.e., the definitions of the terms in the glossary) and their attributes are visible to the users and reviewed. We describe in detail the further refinement of the analysis model in Chapter 5, *Analysis*.

Many heuristics have been proposed in the literature for identifying objects. Here are a selected few:

Heuristics for identifying initial analysis objects

- terms that developers or users need to clarify in order to understand the use case
- recurring nouns in the use cases (e.g., Incident)
- real-world entities that the system needs to keep track of (e.g., FieldOfficer, Resource)
- real-world processes that the system needs to keep track of (e.g., EmergencyOperationsPlan)
- use cases (e.g., ReportEmergency)
- data sources or sinks (e.g., Printer)
- interface artifacts (e.g., Station)
- *Always use application domain terms.*

During requirements elicitation, participating objects are generated for each use case. If two use cases refer to the same concept, the corresponding object should be the same. If two objects share the same name and do not correspond to the same concept, one or both concepts are renamed to acknowledge and emphasize their difference. This consolidation eliminates any ambiguity in the terminology used. For example, Table 4-2 depicts the initial participating objects we identified for the ReportEmergency use case.

Table 4-2 Participating objects for the ReportEmergency use case

Dispatcher	Police officer who manages Incidents. A Dispatcher opens, documents, and closes incidents in response to emergency reports and other communication with FieldOfficers. Dispatchers are identified by badge numbers.
EmergencyReport	Initial report about an Incident from a FieldOfficer to a Dispatcher. An EmergencyReport usually triggers the creation of an Incident by the Dispatcher. An EmergencyReport is composed of a emergency level, a type (fire, road accident, or other), a location, and a description.
FieldOfficer	Police or fire officer on duty. A FieldOfficer can be allocated to at most one Incident at a time. FieldOfficers are identified by badge numbers.
Incident	Situation requiring attention from a FieldOfficer. An Incident may be reported in the system by a FieldOfficer or anybody else external to the system. An Incident is composed of a description, a response, a status (open, closed, documented), a location, and a number of FieldOfficers.

Once participating objects are identified and consolidated, the developers can use them as a checklist for ensuring the set of identified use cases is complete.

Heuristics for cross-checking use cases and participating objects

- Which use cases create this object (i.e., during which use cases are the values of the object attributes entered in the system)? Which actors can access this information?
- Which use cases modify and destroy this object (i.e., which use cases edit or remove this information from the system)? Which actor can initiate these use cases?
- Is this object needed (i.e., is there at least one use case that depends on this information?)

When developers identify new use cases, they describe and integrate the new use case into the model, following the process we described before. Often, in the requirements elicitation activity, shifting perspectives introduces modifications in the system specification (e.g., finding new participating objects triggers the addition of new use cases; the addition of new use cases triggers the addition or refinement of new participating objects). This instability should be anticipated and encourage shifting perspectives. For the same reasons, time-consuming tasks, such as the description of exceptional cases and refinements of the user interfaces, should be postponed until the set of use cases is stable.

4.4.7 Identifying nonfunctional requirements

Nonfunctional requirements describe user-visible aspects of the system that are not directly related to the functional behavior of the system. Nonfunctional requirements span a number of issues, from user interface look and feel to response time requirements to security

issues. Nonfunctional requirements are defined at the same time as functional requirements are, because they have as much impact on the development and cost of the system.

For example, consider a mosaic display that an air traffic controller uses to track planes. A mosaic display system compiles data from a series of radar and databases (hence the term “mosaic”) into a summary display indicating all aircraft in a certain area, including their identification, speed, and altitude. The number of aircraft such a system can display constrains the performance of the air traffic controller and the cost of the system. If the system can only handle a few aircraft simultaneously, the system cannot be used at busy airports. On the other hand, a system able to handle a large number of aircraft is more costly and more complex to build.

Nonfunctional requirements can be elicited by investigating the following issues.

- **User interface and human factors.** What kind of interface should the system provide? What is the level of expertise of the users?
- **Documentation.** What level of document is required? Should only user documentation be provided? Should there be technical documentation for maintainers? Should the development process be documented?
- **Hardware considerations.** Are there hardware compatibility requirements? Will the system interact with other hardware systems?
- **Performance characteristics.** How responsive should the system be? How many concurrent users should it support? What is a typical or extreme load?
- **Error handling and extreme conditions.** How should the system handle exceptions? Which exceptions should the system handle? What is the worst environment in which the system is expected to perform? Are there safety requirements on the system?
- **Quality issues.** How reliable/available/robust should the system be? What is the client’s involvement in assessing the quality of the system or the development process?
- **System modifications.** What is the anticipated scope of future changes? Who will perform the changes?
- **Physical environment.** Where will the system be deployed? Are there external factors such as weather conditions that the system should withstand?
- **Security issues.** Should the system be protected against external intrusions or against malicious users? To what level?
- **Resource issues.** What are the constraints on the resources consumed by the system?

Once all nonfunctional requirements are identified and described, they are prioritized by importance. Although most nonfunctional requirements are highly desirable, some of them need to be met in order for the system to operate correctly.

4.5 Managing requirements elicitation

In the previous section, we described the technical issues of modeling a system in terms of use cases. Use case modeling by itself, however, does not constitute requirements elicitation. Even after they become expert use case modelers, developers still need to elicit requirements from the users and converge onto an agreement with the client. In this section, we describe methods for eliciting information from the users and negotiating an agreement with a client. In particular, we describe:

- eliciting requirements from users: Knowledge Analysis of Tasks (KAT) (Section 4.5.1)
- negotiating a specification with clients: Joint Application Design (JAD) (Section 4.5.2)
- validating requirements: usability testing (Section 4.5.3)
- documenting requirements elicitation (Section 4.5.4)

4.5.1 Eliciting information from users: Knowledge Analysis of Tasks

Task analysis originated in the United States and the United Kingdom in the 1950s and 1960s [Johnson, 1992]. Initially, task analysis was not concerned with requirements or system design. Task analysis was used to identify how people should be trained. In the United States, the military was primarily interested in task analysis to decrease the cost of training. In the United Kingdom, the Department of Trade and Industry was interested in task analysis for developing methods to enable people to move across industries. More recently, task analysis has become important in the field of Human Computer Interaction (HCI) for identifying and describing the user tasks that a system should support.

Task analysis is based on the assumption that it is inefficient to ask users to describe what they do and how they do it. Users usually do not think explicitly about the sequence of tasks that are required to accomplish their work as they have often repeated these tasks many times. Users, when asked how they accomplish their work, would describe, at best, how they are supposed to accomplish it, which may be far from reality. Consequently, task analysis uses observation as an alternative to build an initial task model. This initial task model is then refined by asking the users *why* they accomplish a task a certain way.

The **Knowledge Analysis of Tasks (KAT)** is a task analysis method proposed by Johnson [Johnson, 1992]. It is concerned with collecting data from a variety of sources (e.g., protocol analysis, standard procedures, textbooks, interviews), analyzing these data to identify individual elements involved in the task (e.g., objects, actions, procedures, goals, and subgoals), and constructing a model of the overall knowledge used by people accomplishing the task of interest. KAT is an object-oriented analysis technique in that it represents the application domain in terms of objects and actions.

KAT can be summarized by the five following steps:

1. **Identifying objects and actions.** Object and actions associated with objects are identified using similar techniques as object identification in object-oriented analysis, such as

analyzing textbooks, manuals, rule books, reports, interviewing the task performer, observing the task performer.

2. **Identifying procedures.** A procedure is a set of actions, a precondition necessary to triggering the procedure, and a postcondition. Actions may be partially ordered. Procedures are identified by writing scenarios, observing the task performer, asking the task performer to select and order cards on which individual actions are written.
3. **Identifying goals and subgoals.** A goal is a state to be achieved for the task to be successful. Goals are identified through interview during the performance of a task or afterward. Subgoals are identified by decomposing goals.
4. **Identifying typicality and importance.** Each identified element is rated according to how frequently it is encountered and whether it is necessary for accomplishing a goal.
5. **Constructing a model of the task.** The information gathered above is generalized to account for common features across tasks. Corresponding goals, procedures, and objects are related using a textual notation or a graph. Finally, the model is validated with the task performer.

Although task analysis and KAT are not requirements elicitation methods per se (they do not produce a description of the future software system), they can greatly benefit the requirements elicitation activity in several ways:

- During elicitation, they provide techniques for eliciting and describing application domain knowledge, including information such as typicality and importance of specific actions; the end result is understandable by the task performer.
- When defining the boundaries of a system, task models assist in determining which parts of the task should remain manual and which parts should be automated; moreover, the task model may reveal problem areas in the current system.
- When designing the interface of the system, task models serve as a source of inspiration for metaphors understandable by the user [Nielsen & Mack, 1994].

For more information about KAT, the reader is referred to specialized literature [Johnson, 1992].

4.5.2 Negotiating specifications with clients: Joint Application Design

Joint Application Design (JAD) is a requirements method developed at IBM at the end of the 1970s. Its effectiveness lies in that the requirements elicitation work is done in one single workshop session in which all stakeholders participate. Users, clients, developers, and a trained session leader sit together in one room to present their viewpoint, listen to other viewpoints, negotiate, and agree on a mutually acceptable solution. The outcome of the workshop, the final JAD document, is a complete system specification document that includes definitions of data elements, work flows, and interface screens. Because the final document is jointly developed by

the stakeholders (that is, the participants who not only have an interest in the success of the project, but also can make substantial decisions) the final JAD document represents an agreement between users, clients, and developers, and thus minimizes requirements changes later in the development process.

JAD is composed of five activities (summarized in Figure 4-12):

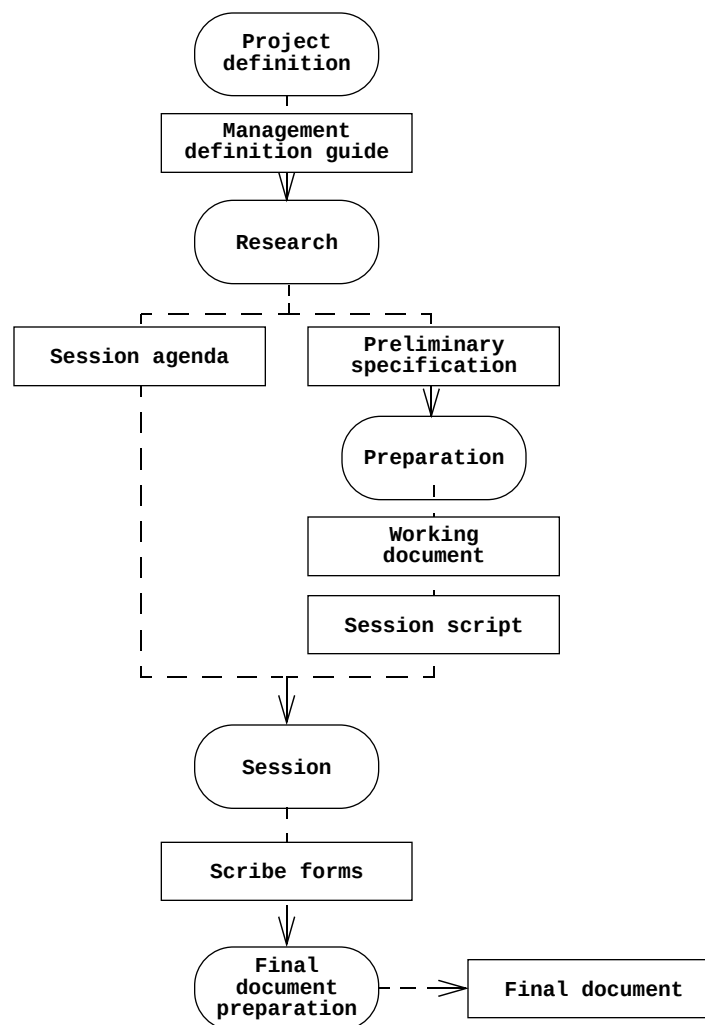


Figure 4-12 Activities of JAD (UML activity diagram). The heart of JAD is the Session activity during which all stakeholders design and agree to a system specification. The activities prior to the Session maximizes its efficiency. The production of the final document ensures that the decisions made during the Session are captured.

1. **Project definition.** During this activity, the JAD facilitator interviews managers and clients to determine the objectives and the scope of the project. The findings from the interviews are collected in the *Management Definition Guide*. During this activity, the JAD facilitator forms a team composed of users, clients, and developers. All stakeholders are represented, and the participants are able to make binding decisions.
2. **Research.** During this activity, the JAD facilitator interviews present and future users, gathers domain information, and describes the work flows. The JAD facilitator also starts a list of issues that will need to be addressed during the session. The primary results of the research activity are a *Session Agenda* and a *Preliminary Specification* listing work flow and system information.
3. **Preparation.** During this activity, the JAD facilitator prepares the session. The JAD facilitator creates a *Working Document*, first draft of the final document, an agenda for the session, and any number of overhead slides or flip charts representing information gathered during the Research activity.
4. **Session.** During this activity, the JAD facilitator guides the team in creating the system specification. A JAD session lasts for 3–5 days. The team defines and agrees on the work flow, the data elements, the screens, and the reports of the system. All decisions are documented by a scribe filling JAD forms.
5. **Final document.** The JAD facilitator prepares the *Final Document*, revising the working document to include all decisions made during the session. The Final Document represents a complete specification of the system agreed on during the session. The Final Document is distributed to the session's participants for review. The participants then meet for a 1-to 2-hour meeting to discuss the reviews and finalize the document.

JAD has been successfully used in IBM and other companies. JAD leverages off group dynamics to improve communication among participants and to accelerate consensus. At the end of a JAD session, developers are more knowledgeable of users needs, and users are more knowledgeable of development trade-offs. Additional gains result from a reduction of redesign activities downstream. Because of its reliance on social dynamics, the success of a JAD session often depends on the qualifications of the JAD facilitator as a meeting facilitator.

4.5.3 Validating requirements: Usability testing

Usability testing tests the user's understanding of the use case model. Usability testing finds problems with the system specification by letting the user explore the system or only part of the system (e.g., the user interface). Usability tests are also concerned with user interface details, such as the look and feel of the user interface, the geometrical layout of the screens, and the hardware. For example, in case of a wearable computer, a usability test might test the ability of the user to issue commands to the system while lying in an awkward position, as in the case of a mechanic looking at a screen under a car while checking a muffler.

Usability tests are based on experiments identifying deficiencies that developers fix iteratively [Rubin, 1994]. The technique for conducting usability tests is based on the classical approach for conducting a controlled experiment. Developers formulate a test objective, which they then test by manipulating selected experimental parameters under controlled conditions. Developers carefully study the values of these parameters to identify statistically significant cause-and-effect relationships. Even though this type of approach could be used for any parameter, usability tests focus on usability parameters, such as the ease to learn the system, the time to accomplish a task, or the rate of errors a user makes when accomplishing a task.

There are two important differences between classical experiments and usability tests. Whereas the classical experimental method is designed to confirm or refute a hypothesis, the goal of usability tests is to obtain qualitative information on how to fix usability problems and how to improve the system. Another difference is the rigor with which the experiments need to be performed. It has been shown that even a series of quick focused tests starting as early as requirements elicitation is extremely helpful. Nielsen uses the term “discount usability engineering” to indicate that a few usability tests are better than none at all [Nielsen, 1993].

There are three types of usability tests:

- **Scenario test.** During this test, one or more users are presented with a visionary scenario of the system. Developers identify how quickly users are able to understand the scenario, how accurately it represents their model of work, and how positively they react to the description of the new system. The selected scenarios should be as realistic and detailed as possible. A scenario test allows rapid and frequent feedback from the user. Scenario tests can be realized as paper mockups² or with a simple prototyping environment, which is often easier to learn than the programming environment used for development. The advantage of scenario tests is that they are cheap to realize and to repeat. The disadvantages are that the user cannot interact directly with the system and that the data are fixed.
- **Prototype test.** During this type of test, one or more users are presented with a piece of software that practically implements the system. A vertical prototype implements a complete use case through the system, and a horizontal prototype presents an interface for most use cases (without providing much or any functionality). The advantages of prototype tests are that they provide a realistic view of the system to the user and that prototypes can be instrumented to collect detailed data. The disadvantages of prototypes are that they are expensive to build and to modify.
- **Product test.** This test is similar to the prototype test except that a functional version of the system is used in place of the prototype. A product test can only be conducted once

2. Storyboarding, a technique from the feature animation industry, consists of sketching a sequence of pictures of the screen at different point in the scenario. The pictures of each scenario are then lined up chronologically against a wall on a board (hence the term “storyboard”). Developers and users can walk around the room when reviewing and discussing the scenarios. Given a reasonably sized room, participants can deal with several hundreds of sketches.

most of the system is developed. It also requires that the system be easily modifiable such that the results of the usability test can be taken into account.

In all three types of tests, the basic elements of usability testing include [Rubin, 1994]:

- development of test objectives
- use of a representative sample of end users
- use of the system in the actual or simulated work environment
- involvement of end users
- controlled, extensive interrogation, and probing of the users by the person performing the usability test
- collection and analysis of quantitative and qualitative results
- recommendations on how to improve the system

Typical test objectives in a usability test address the comparison of two user interaction styles, the identification of the best and the worst features in a scenario or a prototype, the main stumbling blocks, the identification of useful features for novice and expert users, when is help needed, and what type of training information is required.

One of the main problems of usability tests is with enrolling participants. There are several obstacles faced by project managers in selecting real end users [Grudin, 1990]:

- The project manager is usually afraid that customers bypass established technical support organizations and call the developers directly, once they know how to get to them. Once this line of communication is established, developers might be sidetracked too often from doing their assigned jobs.
- Sales personnel do not want developers to talk to their “customers.” Sales people are afraid that they may offend the customer or create dissatisfaction with the current generation of products (which still must be sold).
- The end users do not have time.
- The end users dislike being studied. For example, a automotive mechanic might think that an augmented reality system will put him out of work.

Debriefing the participants is the key to coming to an understanding how to improve the usability of the system being tested. Even though the usability test uncovers and exposes problems, it is often the debriefing session that illustrates why these problems have occurred in the first place. It is important to write recommendations on how to improve the tested components as fast as possible after the usability test is finished, so they can be used by the developers to implement any necessary changes in the system models of the tested component.

For more details on usability testing, the reader is referred to the specialized literature [Rubin, 1994], [Nielsen & Mack, 1994].

4.5.4 Documenting requirements elicitation

The results of the requirements elicitation activity and the analysis activity are documented in the Requirements Analysis Document (RAD). This document completely describes the system in terms of functional and nonfunctional requirements and serves as a contractual basis between the client and the developers. The audience for the RAD includes the client, the users, the project management, the system analysts (i.e., the developers who participate in the requirements), and the system designers (i.e., the developers who participate in the system design). The first part of the document, including use cases and nonfunctional requirements, is written during requirements elicitation. The formalization of the specification in terms of object models is written during analysis. The following is an example template for a RAD:

Requirements Analysis Document

1. Introduction
 - 1.1 Purpose of the system
 - 1.2 Scope of the system
 - 1.3 Objectives and success criteria of the project
 - 1.4 Definitions, acronyms, and abbreviations
 - 1.5 References
 - 1.6 Overview
2. Current system
3. Proposed system
 - 3.1 Overview
 - 3.2 Functional requirements
 - 3.3 Nonfunctional requirements
 - 3.3.1 User interface and human factors
 - 3.3.2 Documentation
 - 3.3.3 Hardware consideration
 - 3.3.4 Performance characteristics
 - 3.3.5 Error handling and extreme conditions
 - 3.3.6 Quality issues
 - 3.3.7 System modifications
 - 3.3.8 Physical environment
 - 3.3.9 Security issues
 - 3.3.10 Resource issues
 - 3.4 Pseudorequirements
 - 3.5 System models
 - 3.5.1 Scenarios
 - 3.5.2 Use case model
 - 3.5.3 Object model
 - 3.5.3.1 Data dictionary
 - 3.5.3.2 Class diagrams
 - 3.5.4 Dynamic models
 - 3.5.5 User interface—navigational paths and screen mock-ups
4. Glossary

The first section of the RAD is an *Introduction*. Its purpose is to provide a brief overview of the function of the system and the reasons for its development, its scope, and references to the development context (e.g., related problem statement, references to existing systems, feasibility studies). The introduction also includes the objectives and success criteria of the project.

The second section, *Current system*, describes the current state of affairs. If the new system will replace an existing system, this section describes the functionality and the problems of the current system. Otherwise, this section describes how the tasks supported by the new system are accomplished now. For example, in the case of SatWatch, the user currently resets her watch whenever she travels across a time zone. Because of the manual nature of this operation, the user occasionally sets the wrong time. In contrast, the SatWatch will continually ensure accurate time within its lifetime. In the case of FRIEND, the current system is paper based: Dispatchers keep track of resource assignments by filling forms. Communication between dispatchers and field officers is radio based. The current system requires a high documentation and management cost that the FRIEND system aims to reduce.

The third section, *Proposed system*, documents the requirements elicitation and the analysis model of the new system. It is divided into five subsections:

- *Overview* presents a functional overview of the system.
- *Functional requirements* describes in natural language the high-level functionality of the system.
- *Nonfunctional requirements* describes user-level requirements that are not directly related to functionality. This includes performance, security, modifiability, error handling, hardware platform, and physical environment.
- *Pseudorequirements* describes design and implementation constraints imposed by the client. This includes the specification of the deployment platform, implementation language, or database management system.
- *System models* describes the scenarios, use cases, object model, and dynamic models describing the system. This section contains the complete functional specification of the system, including mock-ups and navigational charts illustrating the user interface of the system. This section is written during the Analysis activity, described in the next chapter.

The RAD should be written after the use case model is stable, that is, when the number of modifications to the requirements is minimal. The RAD, however, is updated throughout the development process when specification problems are discovered or when the scope of the system is changed. The RAD, once published, is baselined and put under configuration management. The revision history section of the RAD will provide a history of changes as a list of the author responsible for the change, the date of change, and a brief description of the change.

4.6 Exercises

1. Consider your watch as a system and set the time 2 minutes ahead. Write down each interaction between you and your watch as a scenario. Record all interactions, including any feedback the watch provides you.
2. Consider the scenario you wrote in Exercise 1. Identify the actor of the scenario. Next, write the corresponding use case `SetTime`. Include all cases, and include setting the time forward, backward, setting hours, minutes, and seconds.
3. Assume the watch system you described in Exercises 1 and 2 also supports an alarm feature. Describe setting the alarm time as a self-contained use case named `SetAlarmTime`.
4. Examine the `SetTime` and `SetAlarmTime` use cases you wrote in Exercises 2 and 3. Eliminate any redundancy by using an include relationship. Justify why an include relationship is preferable to an extend relationship in this case.
5. Assume the `FieldOfficer` can invoke a `Help` feature when filling an `EmergencyReport`. The `HelpReportEmergency` feature provides a detailed description for each field and specifies which fields are required. Modify the `ReportEmergency` use case (described in Figure 4-11) to include this help functionality. Which relationship should you use to relate the `ReportEmergency` and `HelpReportEmergency`?
6. Below are examples of nonfunctional requirements. Specify which of these requirements are verifiable and which are not.
 - “The system must be usable.”
 - “The system must provide visual feedback to the user within 1 second of issuing a command.”
 - “The availability of the system must be above 95%.”
 - “The user interface of the new system should be similar enough to the old system such that users familiar with the old system can be easily trained to use the new system.”
7. Explain why multiple-choice questionnaires, as a primary means of extracting information from the user, is not effective for eliciting requirements.
8. From your point of view, describe the strengths and weaknesses of users during the requirements elicitation activity. Describe also the strengths and weaknesses of developers during the requirements elicitation activity.
9. Briefly define the term “menu.” Write your answer on a piece of paper and put it upside down on the table together with the definitions of four other students. Compare all five definitions and discuss any substantial difference.

References

- [Bruegge et al., 1994] B. Bruegge, K. O'Toole, & D. Rothenberger, "Design considerations for an accident management system," in M. Brodie, M. Jarke, M. Papazoglou (eds.), *Proceedings of the Second International Conference on Cooperative Information Systems*, University of Toronto Press, Toronto, Canada, pp. 90–100, May 1994.
- [Carroll, 1995] J. M. Carroll (ed.), *Scenario-Based Design: Envisioning Work and Technology in System Development*. Wiley, New York, 1995.
- [Christel & Kang, 1992] M. G. Christel & K. C. Kang, "Issues in requirements elicitation," Software Engineering Institute, Technical Report CMU/SEI-92-TR-12, Carnegie Mellon Univ., Pittsburgh, PA, 1992.
- [Constantine & Lockwood, 1999] L. L. Constantine & L. A. D. Lockwood, *Software for Use*. Addison-Wesley, Reading, MA, 1999.
- [Dumas & Redish, 1998] Dumas & Redish, *A Practical Guide to Usability Testing*. Ablex, New Jersey, 1993.
- [FRIEND, 1994] *FRIEND Project Documentation*, School of Computer Science, Carnegie Mellon Univ., Pittsburgh, PA 1994.
- [Grudin, 1990] J. Grudin, "Obstacles to user involvement in interface design in large product development organizations," *Proceeding IFIP INTERACT'90 Third International Conference on Human-Computer Interaction*, Cambridge, U.K., Aug. 1990.
- [Hammer & Champy, 1993] M. Hammer & J. Champy, *Reengineering The Corporation: a Manifesto For Business Revolution*. Harper Business, New York, 1993.
- [Jackson, 1995] M. Jackson, *Software Requirements & Specifications: A Lexicon of Practice, Principles and Prejudices*. Addison-Wesley, Reading, MA, 1995.
- [Jacobson et al., 1992] I. Jacobson, M. Christerson, P. Jonsson, & G. Overgaard, *Object-Oriented Software Engineering—A Use Case Driven Approach*. Addison-Wesley, Reading, MA, 1992.
- [Jacobson et al., 1999] I. Jacobson, G. Booch, & J. Rumbaugh, *The Unified Software Development Process*. Addison-Wesley, Reading, MA, 1999.
- [Johnson, 1992] P. Johnson, *Human Computer Interaction: Psychology, Task Analysis and Software Engineering*. McGraw-Hill Int., London, 1992.
- [Macaulay, 1996] L. Macaulay, *Requirements Engineering*. Springer Verlag, London, 1996.
- [Nielsen, 1993] J. Nielsen, *Usability Engineering*. Academic, 1993.
- [Nielsen & Mack, 1994] J. Nielsen & R. L. Mack (eds.), *Usability Inspection Methods*. Wiley, 1994.
- [OMG, 1998] Object Management Group, *OMG Unified Modeling Language Specification*. Framingham, MA, 1998. <http://www.omg.org>.
- [Paech, 1998] B. Paech, "The Four Levels of Use Case Description," *4th Int. Workshop on Requirements Engineering: Foundations for Software Quality*, Pisa, June 1998.
- [Rubin, 1994] J. Rubin, *Handbook of Usability Testing*. Wiley, New York, 1994.
- [Wirfs-Brock, 1995] R. Wirfs-Brock, "Design objects and their interactions: A brief look at responsibility-driven design," in *Scenario-Based Design: Envisioning Work and Technology in System Development*. J. M. Carroll (ed.), Wiley, New York, 1995.
- [Wirfs-Brock et al., 1990] R. Wirfs-Brock, B. Wilkerson, & Lauren Wiener, *Designing Object-Oriented Software*. Prentice Hall, Englewood Cliffs, NJ, 1990.
- [Wood & Silver, 1989] J. Wood & D. Silver, *Joint Application Design*[®]. Wiley, New York, 1989.
- [Zultner, 1993] R. E. Zultner. "TQM for Technical Teams," *Communications of the ACM*, vol 36, no. 10, 1993.

5

5.1 Introduction: An optical illusion	132
5.2 An overview of analysis	132
5.3 Analysis concepts	134
5.3.1 Entity, boundary, and control objects	134
5.3.2 Association multiplicity revisited	135
5.3.3 Qualified associations	137
5.3.4 Generalization	138
5.4 Analysis activities: From use cases to objects	139
5.4.1 Identifying entity objects	140
5.4.2 Identifying boundary objects	142
5.4.3 Identifying control objects	144
5.4.4 Modeling interactions between objects: Sequence diagrams	146
5.4.5 Identifying associations	149
5.4.6 Identifying attributes	151
5.4.7 Modeling the nontrivial behavior of individual objects	153
5.4.8 Modeling generalization relationships between objects	153
5.4.9 Reviewing the analysis model	154
5.4.10 Analysis summary	155
5.5 Managing analysis	157
5.5.1 Documenting analysis	157
5.5.2 Assigning responsibilities	158
5.5.3 Communicating about analysis	159
5.5.4 Iterating over the analysis model	161
5.5.5 Client sign-off	162
5.6 Exercises	164
References	165